DEVOPS CAPSTONE · END-TO-END DELIVERY

ClinicDesk

A clinic appointments app — built, tested, scanned, and deployed by pipeline, not by hand.

commit → test → scan → publish → GitOps → cluster → observe

---

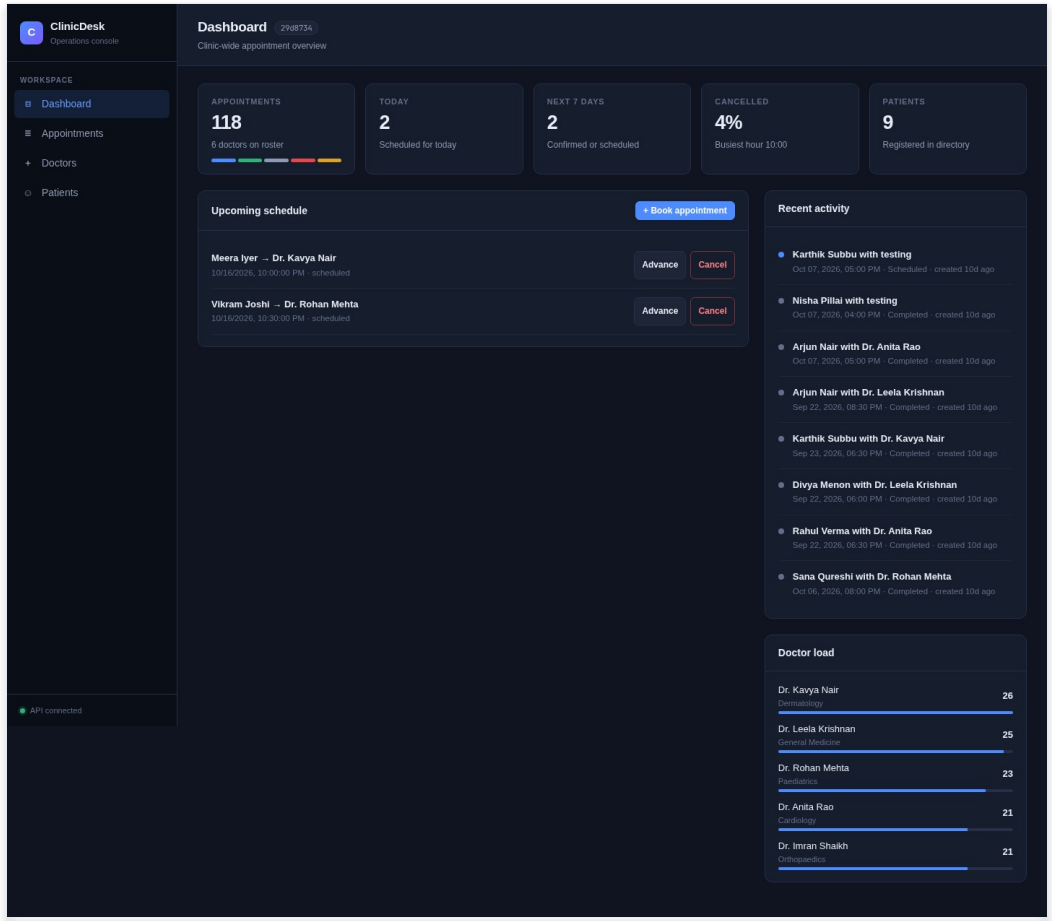
github.com/Datman701/devops_capstone

Terraform · Kubernetes · Helm · GitHub Actions · ArgoCD · Prometheus · Grafana

One small app, one serious pipeline

ClinicDesk manages doctors, patients, and appointment bookings. It is deliberately small — the real deliverable is everything around it: automated builds, security scans, infrastructure, deployment, and observability.

Frontend	React + Vite, served by nginx
Backend	FastAPI · Python 3.12 · REST API + /metrics
Database	PostgreSQL 16 with persistent storage



Every step is automated

A commit travels a fully automated path from developer to a measured, running container.



Gated by design

Nothing reaches the cluster unless it passes tests and the security scan.

Reproducible

The chart deploys the same SHA-tagged images that CI published.

Tools chosen for automation, not novelty

APP	CONTAINERS	CI / CD	REGISTRY
FastAPI · React	**Docker**	**GitHub Actions**	**GHCR**
PostgreSQL 16	multi-stage builds	gated jobs	commit-SHA tags

INFRA	RUNTIME	GITOPS	METRICS
Terraform	**Kubernetes**	**ArgoCD**	**Prometheus**
AWS VPC + EKS	+ Helm chart	declarative sync	+ Grafana

Small images, least privilege

Reviewed Dockerfiles that ship only what runs.

Multi-stage builds

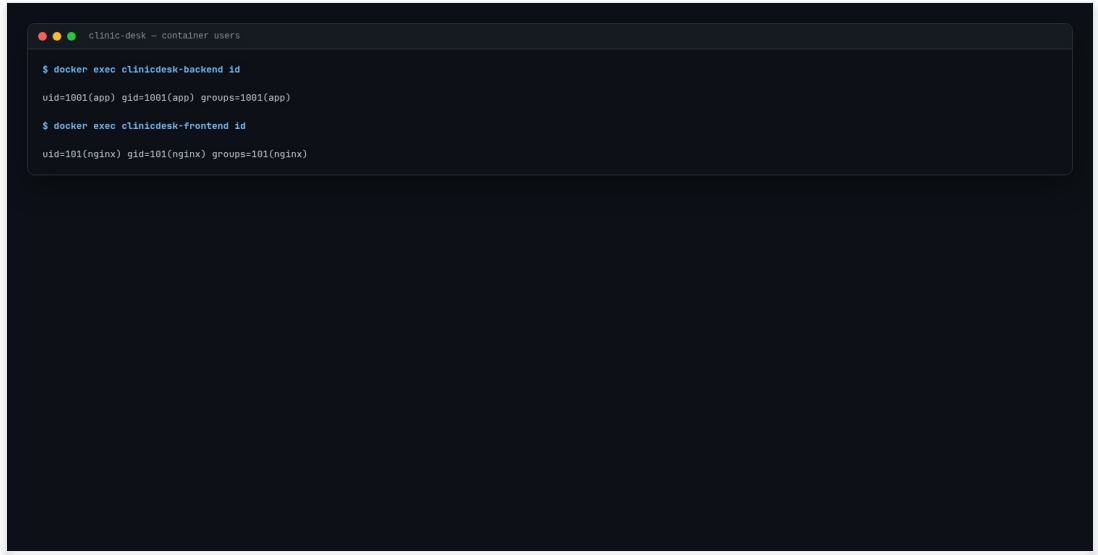
Compilers and build tools never reach the runtime image.

Non-root runtime

Backend runs as uid 1001, frontend as uid 101.

Pinned bases + health checks

Predictable builds and probe-ready containers.



Three gated jobs, one green path

Every push runs the same pipeline; each job blocks the next.

backend-test

85 pytest tests plus ruff lint must pass first.

frontend-build

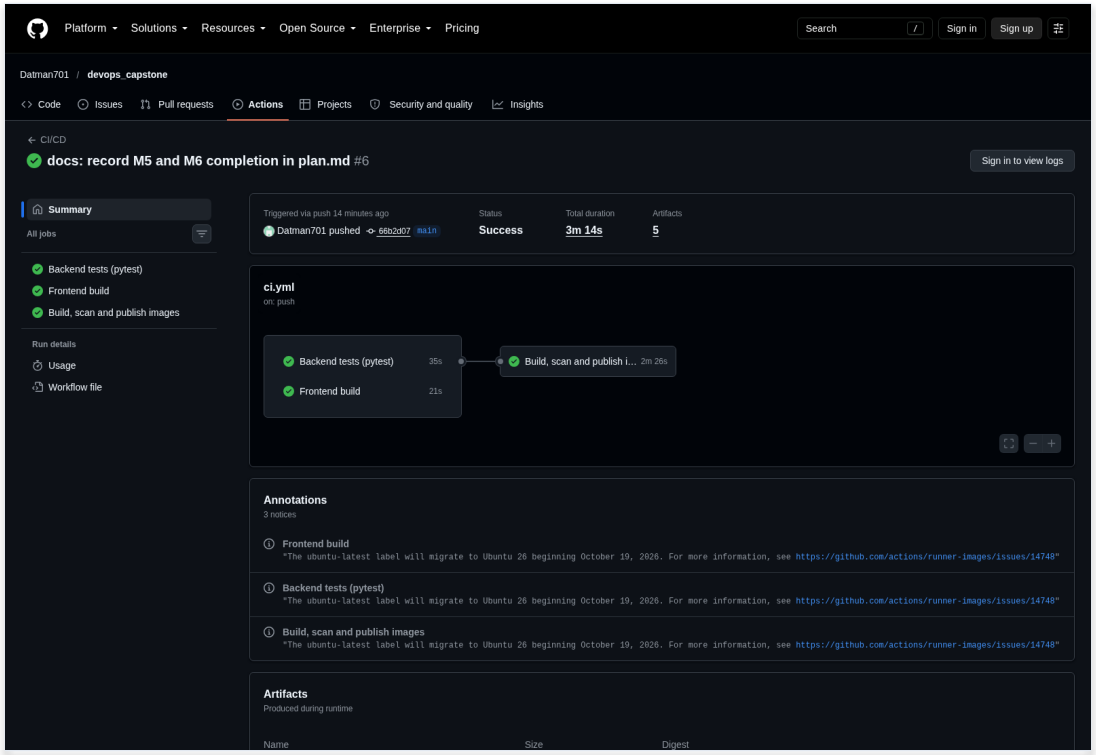
Production bundle is built and checked.

docker

Images are built and pushed to GHCR.

SHA-tagged, never :latest

Each image carries its commit SHA for traceability.



Security is a gate, not an afterthought

Trivy runs inside CI, before anything is published.

Scanned in the pipeline

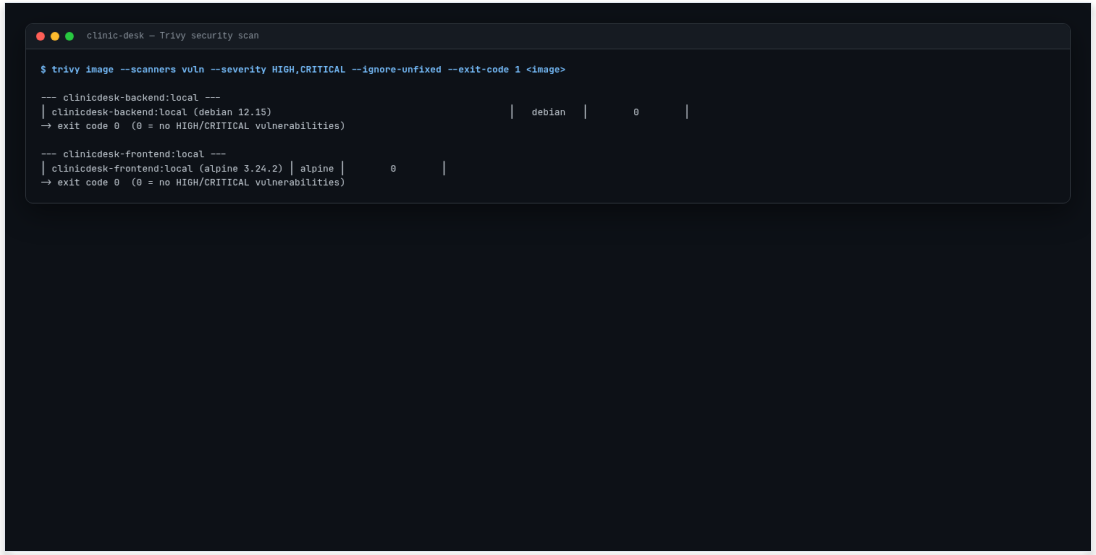
Both images are scanned for known vulnerabilities.

Fail on HIGH / CRITICAL

A serious finding stops the build and the publish.

Clean result shipped

The released images report 0 HIGH / 0 CRITICAL.



Infrastructure defined once, in code

Terraform builds and tears down the AWS foundation.

Networking

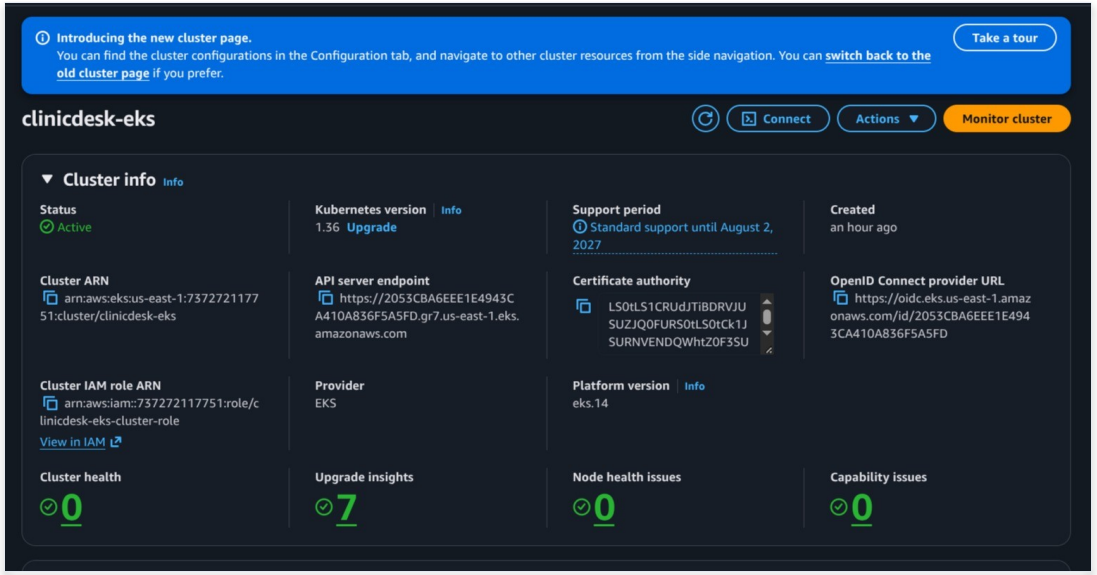
VPC with 2 public + 2 private subnets and an Internet Gateway.

Compute

Amazon EKS 1.36 with a managed worker node group.

27 resources, one plan

Provisioned by terraform apply, removed by destroy.



Declarative deploys that roll, not break

One Helm chart describes the whole application.

A chart, not a script

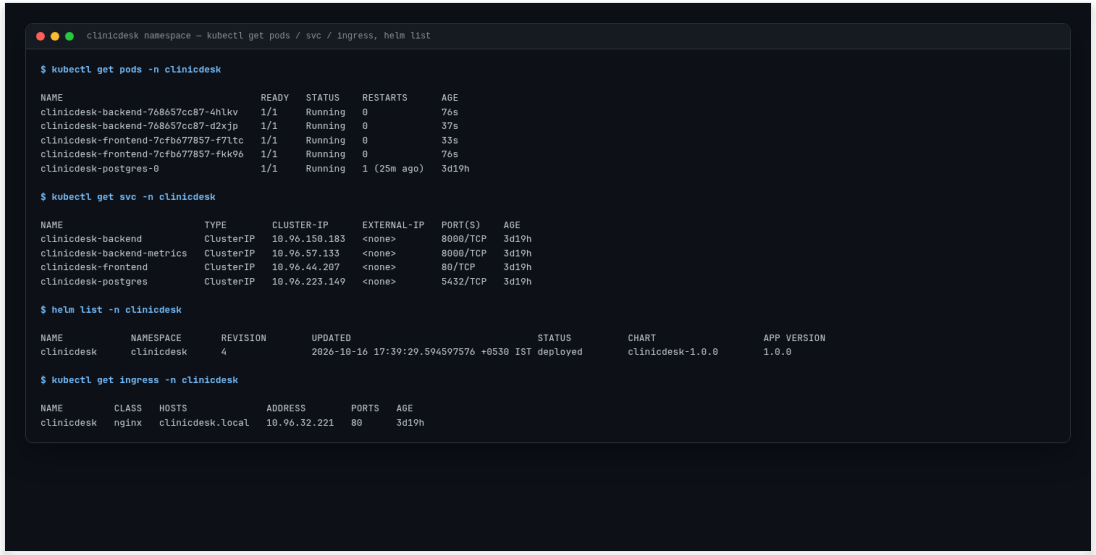
Frontend, backend and PostgreSQL defined as templates.

5 pods running

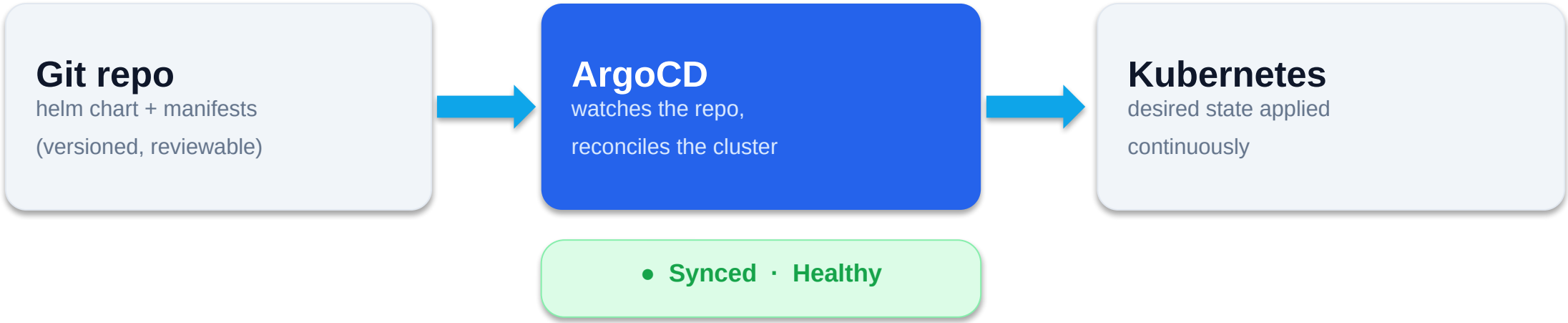
Two backend, two frontend, one database replica.

Zero-downtime updates

Rolling updates with maxUnavailable 0, ClusterIP services, nginx Ingress.



Git is the single source of truth



Declarative, versioned, auditable

The desired cluster state lives in Git alongside the code.

Self-correcting

Drift is detected and reconciled automatically; rollback is a git revert.

If it runs, it is measured

Metrics close the loop from request to container.

Prometheus scraping

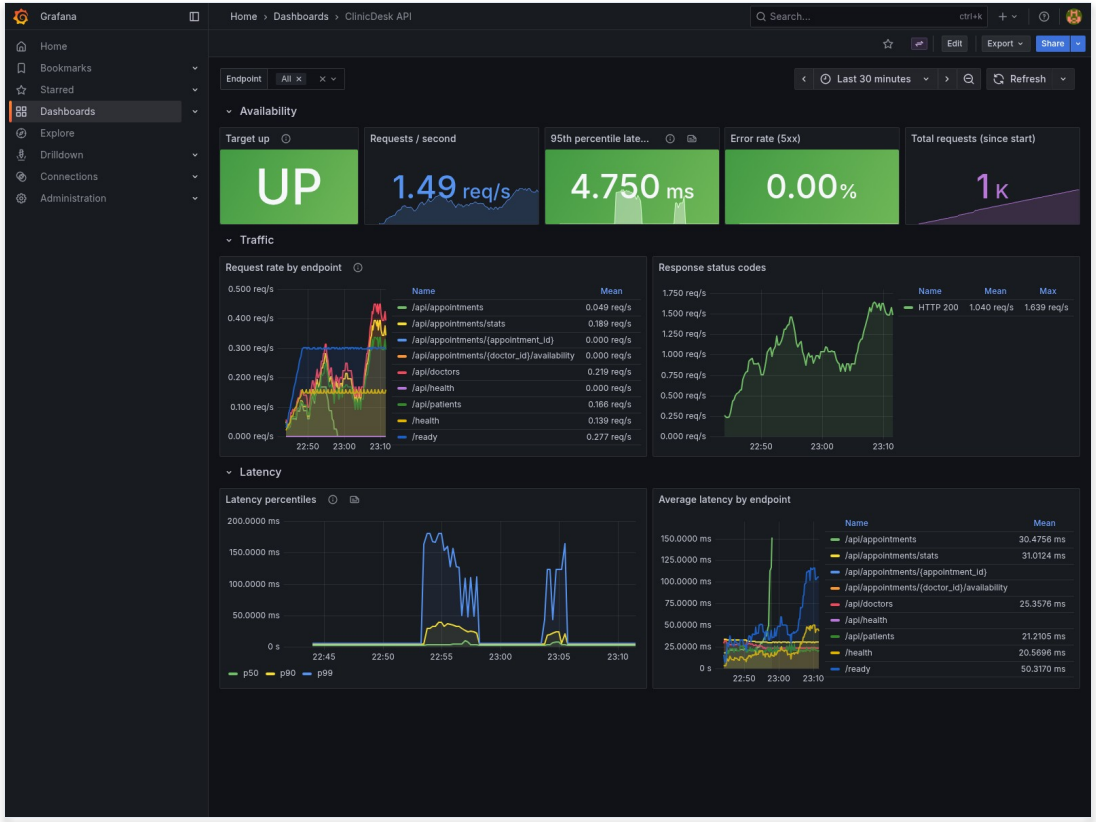
The API exposes /metrics; target clinicdesk-api reports UP.

Grafana dashboard

Nine live panels covering throughput, latency, errors, DB and resources.

Proves the pipeline

Dashboards read the same instance CI deployed.



Proof, not promises

85	0	5	27	9
tests passing	HIGH / CRITICAL	pods running	Terraform resources	Grafana panels

The running app displays its own commit SHA — live proof that source, CI, registry, and cluster agree.

github.com/Datman701/devops_capstone